Mo Farhat
Feb 16, 2026
IT FDN 110 GP Wi 26: Foundations Of Python Programming
Assignment 6 - Functions

Introduction

This document details the steps taken to complete Assignment 06, "Functions, Classes, and Separation of Concerns," for the Introduction to Programming with Python course. The assignment required reviewing module resources on improving scripts using functions, classes, and the separation of concerns programming pattern. The primary goal of the module assignment was to update the menu-driven Python script, building on previous work by demonstrating the use of constants, variables, user input (input()), print statements (print()), a primary while loop for the menu, and conditional logic. The program now manages student registration data using a two-dimensional list of dictionaries for in-memory storage and achieves data persistence by using the json module to read from and save data to a file named "Enrollments.json," while incorporating structured error handling (try-except) that now utilizes a dedicated error handling function.

The core addition in this assignment is the implementation of functions and classes to better organize the code and apply the Separation of Concerns design pattern.

Assignment Completion Steps

The completion of this assignment followed the tasks outlined in the "Mod06-Assignment.docx" document.

1. Read Module Notes and Watch Module Videos

The initial step involved reading the Module 06 Notes document and watching all associated module videos to establish a foundational understanding of functions, classes, and separation of concerns in Python. Key concepts included: functions, parameters, arguments, and return values; the difference between global and local variables; the difference between a function and a class; how to use classes to organize functions; how functions aid in the Separation of Concerns pattern.

2. Watch Assignment Videos and Read About Module Topics

In addition to the core module materials, supplemental videos and the specified external article on functions were reviewed to cover practical topics for this assignment.

3. Create the Program (Assignment06.py)

I created a Python program named Assignment06.py as specified in the assignment document, starting from the Assignment06-Starter.py file. The program script was developed and tested using the PyCharm IDE and console, and includes the following features to meet the acceptance criteria:

- **File Name and Script Header:** The file was named "Assignment06.py," and the script header was updated with my name and the current date.

Assignment06.py

```python
# ------------------------------------------------------------------------------------------ #
# Title: Assignment06.py
# Desc: This assignment demonstrates using functions
# with structured error handling
# Change Log: (Who, When, What)
#   mofarhat, 2/16/2026, Created Script from Assignment06-Start.py
# ------------------------------------------------------------------------------------------ #
```

- **Imports:** The json module was imported.

```python

import json

```

- **Define Constants:** The required constants were defined and remain unchanged throughout the program:
  - MENU: str set to the four-choice menu string for registration.
  - FILE_NAME: str set to "Enrollments.json".

```python
# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------
'''
FILE_NAME: str = "Enrollments.json"
```

- **Define Data Variables:** The required variables were initialized, including:
  - menu_choice: str was initialized to an empty string.
  - students: list was initialized to an empty list ([]) to serve as the two-dimensional list of dictionary rows.

```python
# Define the Data Variables and constants
students: list = []  # a table of student data
menu_choice: str = ''  # Hold the choice made by the user.
```

- **Classes:** The program includes two classes, FileProcessor and IO, both including descriptive document strings.

```python
class FileProcessor:
    """
    A collection of processing layer functions that work with Json files

    ChangeLog: (Who, When, What)
    mofarhat, 2/16/2026 Created class
    """
```

```python
class IO:
    """
    A collection of presentation layer functions that manage user input and output

    ChangeLog: (Who, When, What)
        mofarhat, 2/16/2026, Created Class
        mofarhat, 2/16/2026, Added output menu function
        mofarhat, 2/16/2026, Added output error messages function
    """
```

- **Functions:** All functions include descriptive document strings and use the @staticmethod decorator. The required functions were implemented:
    - output_error_messages(message: str, error: Exception = None)

```python
    @staticmethod
    def output_error_messages(message: str, error: Exception = None):
        """ This function displays a custom error message to the user

        ChangeLog: (Who, When, What)
        mofarhat, 2/16/2026, Created function

        :return: None
        """
        print(message, end="\n\n")
        if error is not None:
            print("-- Technical Error Message -- ")
            print(error, error.__doc__, type(error), sep='\n')
```

    - output_menu(menu: str)

```python
    @staticmethod
    def output_menu(menu: str):
        """ This function displays the menu of choices to the user

        ChangeLog: (Who, When, What)
        mofarhat, 2/16/2026, Created function

        :return: None
        """
        print()  # Adding extra space to make it look nicer.
        print(menu)     # Present the menu of choices
        print()  # Adding extra space to make it look nicer.
```

- input_menu_choice()

```python
    @staticmethod
    def input_menu_choice():
        """ This function gets a menu choice from the user

        ChangeLog: (Who, When, What)
        mofarhat, 2/16/2026, Created function

        :return: string with the users choice
        """
        choice = "0"
        try:
            choice = input("Enter your menu choice number: ")
            if choice not in ("1", "2", "3", "4"):  # Note these are strings
                raise Exception("Please, choose only 1, 2, 3, or 4")
        except Exception as e:
            IO.output_error_messages(e.__str__())  # Not passing e to avoid the technical message
        return choice
```

- output_student_courses(student_data: list)

```python
    @staticmethod
    def output_student_courses(student_data: list):
        """ This function displays a string of comma-separated values for each row in the students variable.

        ChangeLog: (Who, When, What)
        mofarhat, 2/16/2026, Created function

        :return: None
        """
        print("-" * 50)
        if len(student_data) > 0:
            for student in student_data:
                print(f'{student["FirstName"]},{student["LastName"]},{student["CourseName"]}')
        else:
            print("No data to display.")
        print("-" * 50)
```

- input_student_data(student_data: list)

```python
    @staticmethod
    def input_student_data(student_data: list):
        """ This function gets input from the user on Student and Course names

        ChangeLog: (Who, When, What)
        mofarhat, 2/16/2026, Created function

        :return: none
        """
        try:
            student_first_name = input("Enter the student's first name: ")
            if not student_first_name.isalpha():
                raise ValueError("The first name should not contain numbers.")
            student_last_name = input("Enter the student's last name: ")
            if not student_last_name.isalpha():
                raise ValueError("The last name should not contain numbers.")
            course_name = input("Please enter the name of the course: ")
            new_data = {"FirstName": student_first_name,
                        "LastName": student_last_name,
                        "CourseName": course_name}
            student_data.append(new_data)
            print(f"You have registered {student_first_name} {student_last_name} for {course_name}.")
        except ValueError as e:
            IO.output_error_messages("That value is not the correct type of data!", e)
        except Exception as e:
            IO.output_error_messages("Error: There was a problem with your entered data.", e)
        return student_data
```

- read_data_from_file(file_name: str, student_data: list)

```python
    @staticmethod
    def read_data_from_file(file_name: str, student_data: list):
        """ This function reads data from a json file into a list of dictionary rows

            ChangeLog: (Who, When, What)
            mofarhat, 2/16/2026, Created function

            :return: list
        """
        try:
            file = open(file_name, "r")
            new_data = json.load(file)
            for student in new_data:
                student_data.append(student)
            file.close()
            return student_data
        except FileNotFoundError as e:
            IO.output_error_messages("Text file must exist before running this script!", e)
        except Exception as e:
            IO.output_error_messages("There was a non-specific error!", e)
        finally:
            if file.closed == False:
                file.close()
```

- write_data_to_file(file_name: str, student_data: list)

```python
    @staticmethod
    def write_data_to_file(file_name: str, student_data: list):
        """ This function writes data to a json file into a list of dictionary rows

            ChangeLog: (Who, When, What)
            mofarhat, 2/16/2026, Created function
        """
        try:
            file = open(file_name, "w")
            json.dump(student_data, file, indent=2)

            file.close()
            print("The following data was saved to file!")
            for student in student_data:
                print(f'{student["FirstName"]},{student["LastName"]},{student["CourseName"]}')
        except Exception as e:
            IO.output_error_messages("There was an error writing to file", e)
        finally:
            if file.closed == False:
                file.close()
```

- **Program Flow (Menu and Processing):**
  - The program now calls the read_data_from_file function upon startup to load initial data from "Enrollments.json."

```python
# Read the data from file, present the menu and process user input
FileProcessor.read_data_from_file(file_name=FILE_NAME, student_data=students)
```

  - A primary while (True): loop calls the output_menu and input_menu_choice functions to control the menu flow.

```python
while True:
    IO.output_menu(menu=MENU)
    menu_choice = IO.input_menu_choice()
```

  - Menu choices call the appropriate functions:
    - **Menu Choice 1 (Register a Student):** Calls input_student_data to prompt the user and add the new record to the students list.
    - **Menu Choice 2 (Show current data):** Calls output_student_courses to display the comma-separated records in the students list.
    - **Menu Choice 3 (Save data to a file):** Calls write_data_to_file to open "Enrollments.json," write the students list using json.dump(), and close the file.
    - **Menu Choice 4 (Exit the program):** Uses the break statement to end the program.

```python
    # Input user data
    if menu_choice == "1":
        IO.input_student_data(student_data=students)
        continue

    # Present the current data
    elif menu_choice == "2":
        IO.output_student_courses(student_data=students)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
        continue

    # Stop the loop
    elif menu_choice == "4":
        break  # out of the loop
    else:
        print("Please only choose option 1, 2, 3, or 4")

print("Program Ended")
```

- **Error Handling:** Structured error handling with try-except blocks was implemented for four scenarios, and each except block now includes a call to the output_error_messages function:
    - When the file is read into the list of dictionary rows (json.load()).
    - When the user enters a first name (input()).
    - When the user enters a last name (input()).
    - When the dictionary rows are written to the file (json.dump()).

```python
except FileNotFoundError as e:
    IO.output_error_messages("Text file must exist before running this script!", e)
```

```python
except ValueError as e:
    IO.output_error_messages("That value is not the correct type of data!", e)
```

```python
except Exception as e:
    IO.output_error_messages("There was a non-specific error!", e)
```

```python
except Exception as e:
    IO.output_error_messages("There was an error writing to file", e)
```

The program was tested in both PyCharm and directly from the terminal and was confirmed to run correctly, meeting all acceptance and test criteria.

4. Document Knowledge

This document was created to describe the steps and process of completing Assignment 06, saved as Assignment06_MoFarhat.pdf

5. Source Control and Submission

The program files and documentation were posted to the required public GitHub repository and submitted to the course management system:

- **GitHub Repository Creation:** A public repository named "IntroToProg-Python-Mod06" was created on GitHub, accessible on https://github.com/UWFarhat/IntroToProg-Python-Mod06
- **File Posting:** The three required files—the knowledge document (Assignment06_MoFarhat.pdf), the Python script (Assignment06.py), and the data file (Enrollments.json with sample data)—were uploaded to the repository and the changes were committed.
- **Link Posting:** The URL for the new GitHub site was copied into the knowledge document and will be posted to the "Assignment 06 Documents for Review!" discussion board on Canvas for peer review.
- **Final Submission:** The three files were placed into a folder named A06, compressed into a ".zip" file, and uploaded to the class assignment page on Canvas.

Summary

Assignment 06 refactored the student registration application to incorporate functions and classes to achieve a more organized and maintainable code structure, aligning with the Separation of Concerns pattern. The core functionality—using a two-dimensional list of dictionaries for data, reading/writing to a JSON file, and menu-driven execution—was preserved and integrated into the new class and function structure. The structured error handling consolidated error messaging into a dedicated function.